

TD n° 9 – Tableaux à 2 dimensions

Exercice 1 ;

Considérons les déclarations suivantes :

constante entier NB_LIG := 5 ;

constante entier NB_COL := 10;

type t_tablo = **tableau** [NB_LIG,NB_COL] de **entier** ;

1. **Question** - Proposez une procédure pour saisir toutes les valeurs d'un tableau de type t_tablo
2. **Question** - Proposez une procédure pour l'affichage d'un tableau de type t_tablo
3. **Question** - Proposez un programme permettant de tester ces procédures.

Exercice 2 : Matrice carrée :

1. **Question** - Proposez une fonction délivrant la somme des éléments de la diagonale « Nord Ouest –Sud Est » d'un tableau de type t_matrice_carree.

constante entier NB := 10;

type t_matrice_carree = **tableau** [NB,NB] de **entier** ;

2. **Question** - Proposez une procédure qui affiche le triangle supérieur gauche d'un tableau de type t_matrice_carree.

44	63	36	22	48
43	65	82	37	
54	76	24		
34	67			
74				

3. **Question** - Proposez une procédure pour transposer une matrice contenue dans un tableau de type t_matrice_carree (sans utiliser d'autres tableaux).

(1,1)	(1,2)	(1,3)	(1,4)	(1,5)
(2,1)	(2,2)	(2,3)	(2,4)	(2,5)
(3,1)	(3,2)	(3,3)	(3,4)	(3,5)
(4,1)	(4,2)	(4,3)	(4,4)	(4,5)
(5,1)	(5,2)	(5,3)	(5,4)	(5,5)

Exercice 3 :

Considérons les déclarations suivantes :

constante entier NB_LIG := 3 ;

constante entier NB_COL := 4 ;

type t_ligne = **tableau**[NB_LIG] de **entier** ;

type t_colonne = **tableau**[NB_COL] de **entier** ;

type t_matrice = **tableau**[NB_LIG, NB_COL] de **entier** ;

1. **Question** - Proposez la procédure sommeLigneColonne(entF m : matrice, sortF sL : t_ligne, sortF sC : t_colonne) qui calcule la somme de chaque ligne (dans le paramètre sL de type t_ligne) et de chaque colonne (dans le paramètre sC de type t_colonne) d'une matrice m.

	16	13	11	10
15	4	6	3	2
26	6	5	8	7
9	6	2	0	1

Exercice 3

Nous voulons écrire un programme permettant à un utilisateur de connaître la distance qui sépare deux villes bretonnes dont il saisit les noms au clavier.

Pour cela, on mémorise la distance qui sépare des grandes villes bretonnes comme suit :

Les villes seront enregistrées dans un tableau à 1 dimension. Les distances entre villes seront quant à elles mémorisées dans une matrice carrée, comme le montre l'exemple ci-dessous (les valeurs ne sont pas significatives) :

Villes :

Nantes	Rennes	Lannion	Brest	Vannes
1	2	3	4	5

Distances :

	1	2	3	4	5
1	0	130	285	334	125
2	130	0	180	251	119
3	285	180	0	105	167
4	334	251	105	0	199
5	125	119	167	199	0

Remarques :

- la diagonale descendante ne contient que des 0 (la distance de Lannion à Lannion est égale à 0 !)
- le triangle supérieur est identique au triangle inférieur (la distance entre Rennes et Brest est la même qu'entre Brest et Rennes).

Exemple : Nantes est le n° 1 dans le tableau stockant les villes et Vannes le n° 5. La distance entre Nantes et Vannes est donnée par la case (1,5) du tableau stockant les distances, ou par la case (5,1).

1. **Question** - Proposez une définition pour les types de ces 2 tableaux.

2. **Question** - Écrivez en langage algorithmique un programme demandant deux noms de ville, et affichant, si possible, la distance entre ces deux villes (on supposera qu'il existe la procédure remplirDistance () qui complètera les différentes distances entre les villes selon le tableau ci dessus).

Pour cela, vous développerez :

- La fonction rechercheVille() qui retournera l'indice de la ville dans le tableau (ou -1 en cas d'échec)
- La fonction rechercheDistance() qui retournera la distance entre deux villes.

3. **Question** - Écrivez en langage algorithmique un programme utilisant les procédures et fonctions décrites ci-dessus.

Exercice complémentaire 4 : jeu du démineur

Le démineur est un jeu présentant un plateau de dimension N x N, matérialisé par un tableau à 2 dimensions. Le principe est le suivant : le joueur dévoile une à une les cases du plateau dans lequel se trouvent N mines. La partie s'arrête par une victoire si toutes les cases exceptées celles contenant les mines sont dévoilées, par un échec si le joueur a dévoilé un nombre de mines égal au nombre de vies (N/2) dont il dispose au début du jeu, ou lorsqu'il n'y a plus de case à dévoiler en indiquant le nombre de vies restantes.

Un jeu de démineur utilise 2 tableaux de dimension N x N :

- Un **tableau joué** qui renseigne sur les emplacements de mines et de balises : chaque case de ce tableau contient soit une mine, soit une balise indiquant la proximité d'une (ou plusieurs) mine(s). Le contenu de chaque case est matérialisé par un chiffre pouvant avoir une des valeurs suivantes :
 - ✓ 9 pour une mine
 - ✓ un chiffre $n \in [0, 8]$ représentant la balise indiquant combien de mines sont dans des cases adjacentes à la case considérée
- Un **tableau observé** correspondant à ce que le joueur voit au cours du jeu. Au début du jeu ce plateau observé est entièrement « caché », puis il se dévoile au fur et à mesure des cases jouées. Une case

cachée est matérialisée par la valeur -1. Lorsqu'elle est jouée elle contient la valeur de la case identique du tableau joué.

Exemple de plateau joué

0	0	0	0	0	0	0	0	1	9
0	0	1	1	1	0	0	0	2	2
0	1	2	9	1	0	0	1	9	1
0	1	9	2	1	0	0	1	1	1
0	1	1	1	0	0	0	0	0	0
0	0	1	1	1	0	0	0	0	0
0	0	1	9	1	0	1	1	0	0
1	1	2	1	1	0	2	9	3	1
1	9	1	0	0	0	3	9	9	1
1	1	1	0	0	0	2	9	3	1

Exemple de plateau observé au cours du jeu

-1	-1	-1	0	0	-1	-1	-1	-1	-1
-1	0	1	1	1	0	0	0	2	-1
-1	-1	2	9	1	0	-1	1	9	1
-1	-1	-1	2	1	0	-1	-1	1	1
-1	-1	-1	-1	0	0	0	0	0	0
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	0	1	-1	-1	0	1	1	-1	-1
-1	1	2	-1	-1	-1	-1	-1	-1	1
-1	-1	1	0	0	-1	-1	-1	-1	1
-1	-1	1	0	0	-1	-1	-1	-1	1

Nombre de vies restantes : 3

Nombre de mines encore cachées : 8

1. **Question** - Proposez une définition d'un type plateau de démineur en tant que tableau

2. **Question** - Positionnement des mines sur le plateau joué

On considère que l'on dispose d'une procédure `positionMines` calculant aléatoirement la position des N mines sur le plateau, et dont le prototype est le suivant :

procédure `positionMines` (sortF tab: tableau[1..N] de entier).

Cette procédure a pour paramètre de sortie un tableau contenant les N emplacements des mines sur le plateau joué. L'emplacement d'une mine est donné selon un ordre linéaire, c'est-à-dire que les cases sont numérotées de 1 à NxN de gauche à droite et de haut en bas. Ainsi, pour un plateau de jeu de 10x10, le placement 12 correspond à la deuxième ligne en partant du haut et à la deuxième colonne en partant de la gauche.

Proposez une procédure `placerMines` qui prend en paramètre le plateau joué (encore non initialisé), qui fait appel à la procédure `positionMines` pour avoir les positions des mines à placer et qui initialise le plateau joué en y insérant le chiffre 9 à chaque emplacement de mine.

3. **Question** - Calcul des mines adjacentes à une case

Proposez une fonction `minesAdjacentes` prenant en paramètre un plateau joué, un indice de ligne et un indice de colonne d'une case de ce plateau, et qui délivre le nombre de mines adjacentes à la case. Attention à ne pas explorer l'extérieur du plateau.

4. **Question** - Calcul des mines adjacentes sur le tableau joué

Proposez une procédure `remplirPlateau` prenant en paramètre d'entrée/sortie un plateau joué, calculant pour chaque case ne possédant pas une mine, le nombre de mines adjacentes et inscrivant ce nombre dans la case.

Remarque : cet algorithme est simple mais pas optimisé. Pour l'améliorer, il faudrait partir des emplacements des mines et uniquement mettre un chiffre dans les cases adjacentes aux mines.

5. **Question - Initialisation du tableau observé**

On s'intéresse maintenant au plateau observé par le joueur. Proposer une procédure `initPlateauObs` permettant d'initialiser chaque case de ce tableau avec la valeur -1.

6. **Question**

Proposez une procédure `jouer` qui prend en paramètre les deux plateaux, *i.e.* celui des emplacements des mines et des balises ainsi que le plateau observé par l'utilisateur.

Cette procédure demande à l'utilisateur une coordonnée (indice de ligne et indice de colonne) et vérifie qu'il s'agit bien d'une case encore cachée.

Si l'utilisateur tombe sur une mine, il perd une vie sur les N/2 qu'il possède au départ.

Le jeu s'arrête si le joueur n'a plus de vie ou s'il a trouvé tous les emplacements sans mine.